

(Plano de Aula) Plataformas e Integrações: Use o Claude Code no VS Code — Curso Essentials

Platforms and Integrations: Use Claude Code in VS Code — Curso Essentials

Identificação

Campo	Definição
Curso	Plataformas e Integrações: Use o Claude Code no VS Code — Essentials
Natureza	Autoinstrucional (ensino a distância, no próprio ritmo do participante)
Carga horária	90 minutos (distribuídos em 4 módulos de conteúdo e 1 módulo de síntese)
Público-alvo	Docentes do ensino superior iniciantes em Inteligência Artificial; analistas administrativos; alunos de graduação e pós-graduação sem experiência prévia com IA aplicada ao código
Pré-requisitos	Noções elementares de uso do computador (criar pastas, instalar programas, abrir um terminal); familiaridade mínima com qualquer editor de texto ou IDE; acesso à internet e conta ativa na plataforma Claude (claude.ai)
Recursos necessários	Computador pessoal com sistema operacional Windows, macOS ou Linux; VS Code instalado (versão 1.95.0 ou superior); conexão estável à internet; conta Anthropic com chave de API gerada; material-fonte do curso em formato digital

Objetivo Geral

O presente curso tem por finalidade capacitar o participante a instalar, configurar e utilizar a extensão Claude Code no ambiente de desenvolvimento VS Code, de modo que, ao término do percurso formativo, seja capaz de interagir produtivamente com o assistente de Inteligência Artificial para tarefas concretas de seu cotidiano acadêmico ou profissional — tais como análise e explicação de código, refatoração orientada a revisão, geração de documentação e estratégias elementares de depuração —, observando, em todas as etapas, as boas práticas de segurança e privacidade de dados.

Competências a Desenvolver

Concluído o curso, o participante deverá demonstrar capacidade de:

1. Distinguir os diferentes modos de acesso ao assistente Claude (interface web, extensão VS Code e linha de comando), identificando o contexto de uso mais adequado a cada situação.

2. Instalar e configurar a extensão Claude Code no VS Code, incluindo a geração e o armazenamento seguro da chave de API Anthropic.
3. Formular prompts contextualizados — referenciando arquivos abertos e trechos de código — para obter respostas pertinentes às tarefas de análise, refatoração e documentação de código.
4. Avaliar criticamente as sugestões geradas pelo assistente, utilizando o visualizador de diferenças (*diff viewer*) para aceitar, rejeitar ou solicitar ajustes antes de aplicar qualquer alteração ao código-fonte.

Estrutura e Sequência dos Módulos

Os módulos foram organizados em progressão cumulativa: cada módulo pressupõe os conceitos introduzidos no anterior e acrescenta uma camada nova de habilidade. O participante que concluir os quatro módulos de conteúdo em sequência estará apto a integrar todos os aprendizados na atividade integradora do módulo de síntese.

Módulo	Título	Referência (material-fonte)	Tempo estimado
1	O Ecossistema Claude: o que é, o que muda e o que não sai da sua máquina	Aulas 1 e 10 do material-fonte	20 min
2	Instalação e Configuração da Extensão Claude Code no VS Code	Aula 2 do material-fonte	20 min
3	Primeiras Interações: como formular prompts com contexto de arquivo	Aulas 3 e 4 do material-fonte	20 min
4	Refatoração Assistida: o ciclo Perguntar → Planejar → Revisar → Aceitar	Aulas 5 e 8 do material-fonte	20 min
5	Síntese e Aplicação Integrada	Aula 12 do material-fonte	10 min

Módulo 1 — O Ecossistema Claude: o que é, o que muda e o que não sai da sua máquina

Este é o ponto de partida do curso. Antes de instalar qualquer ferramenta, convém compreender o que ela é, como ela se encaixa no fluxo de trabalho cotidiano e quais são seus limites de segurança — fundamento que sustentará todos os módulos seguintes.

Objetivos de Aprendizagem

Ao final deste módulo, o participante deverá ser capaz de:

1. Descrever, com suas próprias palavras, o que é o Claude Code e de que forma ele se diferencia do uso da interface web do Claude.
2. Identificar quais dados são transmitidos aos servidores da Anthropic durante o uso da extensão e quais permanecem armazenados localmente na máquina do usuário.
3. Reconhecer categorias de dados que **não** devem ser compartilhados com o assistente (credenciais, informações pessoais identificáveis, dados sensíveis).
4. Selecionar o modo de acesso ao Claude — interface web ou extensão VS Code — mais adequado a um dado tipo de tarefa.

Texto Descritivo

O Claude é um assistente de Inteligência Artificial desenvolvido pela Anthropic. Em termos simples, trata-se de um programa capaz de compreender linguagem natural — ou seja, frases escritas da forma como as pessoas normalmente se comunicam — e de responder a perguntas, gerar texto, analisar problemas e auxiliar na escrita e na revisão de código de programação. O acesso a esse assistente pode ser feito de três maneiras principais: pela interface web (acessível em qualquer navegador), pela extensão Claude Code instalada diretamente no editor de código VS Code e pela ferramenta de linha de comando (CLI), destinada a usos mais avançados. Este curso concentra-se na segunda modalidade.

O VS Code — sigla de *Visual Studio Code* — é um editor de código-fonte desenvolvido pela Microsoft, amplamente utilizado por programadores, cientistas de dados e demais profissionais que trabalham com arquivos de texto estruturado. Trata-se, em termos simples, de um bloco de notas muito mais sofisticado, capaz de colorir palavras-chave de linguagens de programação, integrar um terminal e comportar centenas de extensões — pequenos programas adicionais que ampliam suas funcionalidades. O Claude Code é uma dessas extensões.

A principal vantagem da extensão sobre a interface web reside no **contexto automático**: quando o participante formula uma pergunta no painel do Claude Code, o assistente pode "ver" o conteúdo do arquivo que está aberto no editor, bem como trechos de código eventualmente selecionados pelo usuário. Na interface web, esse contexto precisaria ser copiado e colado manualmente a cada interação, o que é mais trabalhoso e propenso a erros. Pode-se comparar essa diferença à distinção entre explicar um problema a alguém que está ao seu lado olhando para a mesma tela e explicar o mesmo problema por escrito a alguém que nunca viu o material.

É essencial, no entanto, compreender o modelo de segurança da extensão. Quando o participante formula uma pergunta, o texto dessa pergunta e o conteúdo dos arquivos referenciados são enviados aos servidores da Anthropic para processamento — da mesma forma que ocorre ao enviar uma mensagem de e-mail pela internet. O que **não** é enviado: a chave de API (armazenada localmente pelo sistema operacional), o histórico de conversas (salvo na pasta `~/.anthropic/` da máquina local) e os demais arquivos do projeto que não foram abertos nem mencionados. Por consequência, cumpre observar que determinadas categorias de informação jamais devem ser incluídas em prompts enviados ao assistente: senhas, tokens de autenticação, chaves de API de terceiros, números de documentos pessoais, dados financeiros e quaisquer informações cobertas por legislações de proteção de dados (como a LGPD brasileira ou o GDPR europeu).

A tabela a seguir, adaptada do material-fonte, sintetiza as diferenças entre os três modos de acesso e orienta a escolha adequada conforme o tipo de tarefa:

Aspecto	Interface Web	Extensão VS Code	CLI (Linha de Comando)
Acesso a arquivos locais	Não (exige cópia manual)	Sim (automático, arquivos abertos)	Sim (por caminho completo)
Contexto do projeto	Não	Sim	Sim
Integração ao editor	Não	Sim	Sim (terminal)
Ideal para	Perguntas teóricas, brainstorming	Trabalho em projeto ativo	Automação e scripts

Exercício Prático

Atividade de Classificação de Dados (10 minutos)

Leia os itens abaixo e classifique cada um como **Seguro** (pode ser enviado ao assistente), **Atenção** (requer cuidado e anonimização prévia) ou **Nunca enviar** (proibido em qualquer circunstância):

- Trecho de código Python que calcula a média de uma lista de números inteiros.
- Log de erro contendo o endereço de e-mail de um cliente: `KeyError: 'user@empresa.com.br'`.
- Senha de banco de dados: `db_password = "minha_senha_secreta_123"`.
- Função JavaScript que valida o formato de um CEP.
- Número de CPF de um usuário presente em uma constante de teste: `CPF = "123.456.789-00"`.
- Estrutura de pastas de um projeto sem dados sensíveis.

Gabarito e Fundamentação:

Item	Classificação	Justificativa
1	✅ Seguro	Código genérico, sem dados pessoais ou credenciais
2	⚠️ Atenção	Dado de cliente (e-mail); deve ser anonimizado antes do envio (ex.: substituir por <code>user@example.com</code>)

3	✗ Nunca enviar	Credencial de acesso; sua exposição compromete a segurança do sistema
4	✓ Seguro	Código de validação de formato, sem dados reais
5	✗ Nunca enviar	Número de documento pessoal; dado pessoal protegido por legislação
6	✓ Seguro	Metadado de projeto, sem informação sensível

Critério de êxito: Acertar ao menos 5 das 6 classificações, com correta identificação dos itens 3 e 5 como "Nunca enviar".

Mão na massa 🛠️

Utilize o prompt base abaixo, substituindo os colchetes pelo assunto de sua escolha, para praticar a distinção entre os modos de acesso ao Claude:

Estou iniciando um projeto sobre [DESCREVA SEU TEMA OU ÁREA DE ATUAÇÃO].
Tenho uma tarefa que envolve [DESCREVA A TAREFA: análise de texto, organização de dados, revisão de código, etc.].
Com base nas diferenças entre a interface web do Claude e a extensão Claude Code no VS Code,
qual dos dois modos de acesso seria mais adequado para essa tarefa e por quê?

Exemplo de preenchimento: "Estou iniciando um projeto sobre análise de dados educacionais. Tenho uma tarefa que envolve revisar um script Python que processa notas de alunos..."

Módulo 2 — Instalação e Configuração da Extensão Claude Code no VS Code

Tendo compreendido o que é o Claude Code e seus limites de segurança (Módulo 1), o foco se desloca agora para a instalação efetiva da extensão e a configuração da chave de API — etapas técnicas sem as quais nenhuma interação com o assistente é possível.

Objetivos de Aprendizagem

Ao final deste módulo, o participante deverá ser capaz de:

1. Verificar se o ambiente (versão do VS Code, permissões de rede e políticas institucionais) atende aos requisitos mínimos para a instalação da extensão.
2. Instalar a extensão Claude Code a partir do marketplace do VS Code, identificando a publicação oficial da Anthropic.
3. Gerar uma chave de API no console da Anthropic e configurá-la na extensão de forma segura, sem inseri-la em arquivos versionados.
4. Realizar o teste de verificação de instalação e identificar os passos de resolução dos problemas mais comuns.

Texto Descritivo

Antes de iniciar qualquer instalação, recomenda-se verificar se o ambiente de trabalho atende aos pré-requisitos técnicos. O VS Code deve estar na versão 1.95.0 ou superior — para confirmar, basta acessar o menu *Help* → *About* (Windows e Linux) ou *Code* → *About* (macOS) e observar o número exibido na janela de informações. Caso a versão seja inferior, a atualização pode ser obtida gratuitamente no site oficial do VS Code. Convém ainda verificar se a máquina utilizada é pessoal ou gerenciada por uma instituição: em máquinas de laboratório ou corporativas, pode haver restrições de instalação de extensões ou bloqueios de firewall para o domínio `api.anthropic.com`. Nesse caso, é indispensável consultar a equipe de TI responsável antes de prosseguir.

A instalação da extensão é realizada diretamente no painel de extensões do VS Code, acessível pelo atalho `Ctrl + Shift + X` (Windows e Linux) ou `Cmd + Shift + X` (macOS). No campo de pesquisa, deve-se digitar "claude code" e localizar a extensão publicada pela **Anthropic Inc.** — detalhe importante para evitar a instalação de versões não oficiais. Após clicar em *Install* e aguardar o carregamento (geralmente entre 30 e 60 segundos), pode ser necessário recarregar a janela do editor, o que é feito por meio do comando `Developer: Reload Window`, acessível pela Paleta de Comandos (`Ctrl + Shift + P`).

 **Captura de tela sugerida:** Painel de extensões do VS Code com o resultado da busca por "claude code", destacando o botão *Install* e o nome do publicador "Anthropic Inc."

Após a instalação, é necessário vincular a extensão a uma conta Anthropic por meio de uma **chave de API** — uma sequência alfanumérica única que funciona como credencial de acesso ao serviço, de modo análogo a uma senha, porém destinada a aplicações em vez de pessoas. A chave é gerada no console da Anthropic (acessível em `https://console.anthropic.com`), na seção *API Keys*, onde se deve clicar em *Create API Key*

e atribuir um nome descritivo (por exemplo, "VS Code — Máquina Pessoal"). É imprescindível copiar e guardar a chave imediatamente, pois ela é exibida apenas uma vez pela plataforma.

A inserção da chave no VS Code é feita pelo método de prompt automático: ao clicar no ícone do Claude Code (identificado por um ícone de raio ⚡ na barra lateral esquerda) ou ao executar o comando `Claude Code: Sign In` pela Paleta de Comandos, uma caixa de diálogo solicitará a chave. Após a inserção, o sistema operacional armazenará a credencial de forma segura — no gerenciador de credenciais do Windows, no Keychain do macOS ou no sistema equivalente do Linux — sem que a chave precise ser escrita em nenhum arquivo do projeto.

 **Captura de tela sugerida:** Caixa de diálogo de inserção da chave de API exibida após o comando "Claude Code: Sign In", mostrando o campo de texto para colagem da chave.

Cumpra registrar uma prática que deve ser evitada em absoluto: **jamais inserir a chave de API em arquivos de código ou de configuração que sejam versionados pelo Git**. Um arquivo `.env` contendo a chave, se adicionado ao repositório, expõe a credencial publicamente — o que pode resultar em uso indevido e em cobranças inesperadas na conta Anthropic. A prática correta é adicionar o arquivo `.env` ao arquivo `.gitignore` do projeto, garantindo que ele nunca seja incluído em um *commit*.

Exercício Prático

Verificação de Ambiente e Instalação (15 minutos)

Execute os passos abaixo e registre o resultado de cada um:

1. Abra o VS Code e acesse *Help* → *About* (Windows/Linux) ou *Code* → *About* (macOS). Anote a versão exibida.
2. Confirme se a versão é igual ou superior a 1.95.0. Se não for, realize a atualização antes de prosseguir.
3. Abra o painel de extensões (`Ctrl + Shift + X` ou `Cmd + Shift + X`) e pesquise "claude code".
4. Localize a extensão publicada pela Anthropic Inc. e clique em *Install*.
5. Aguarde a conclusão e recarregue a janela, se solicitado.
6. Acesse `https://console.anthropic.com`, gere uma chave de API e insira-a na extensão pelo comando `Claude Code: Sign In`.
7. No painel do Claude Code, digite: "Hello Claude! What is 2+2?" e confirme que uma resposta é recebida.

Critério de êxito: O assistente responde à pergunta do passo 7 sem exibir mensagem de erro de autenticação. O ícone de raio ⚡ aparece na barra lateral esquerda do VS Code.

Mão na massa

Com a extensão instalada e configurada, utilize o prompt base abaixo para testar a primeira interação contextualizada:

```
Olá! Acabo de instalar o Claude Code no VS Code.  
Estou trabalhando na área de [DESCREVA SUA ÁREA: administração, docência, análise de dados, etc.].  
Poderia me dar um exemplo simples de como poderia usar este assistente para ajudar nas minhas tarefas diárias  
relacionadas a [TAREFA ESPECÍFICA]?
```

Módulo 3 — Primeiras Interações: como formular prompts com contexto de arquivo

Com a extensão instalada e funcionando (Módulo 2), o foco se desloca agora para o uso produtivo da ferramenta — especificamente, para a arte de formular perguntas (denominadas *prompts*) que levem o assistente a fornecer respostas úteis e precisas, aproveitando o contexto automático dos arquivos abertos no editor.

Objetivos de Aprendizagem

Ao final deste módulo, o participante deverá ser capaz de:

1. Formular prompts contextualizados que referenciem explicitamente o arquivo aberto e o problema a ser resolvido, em contraste com prompts vagos e genéricos.
2. Utilizar a sintaxe `@nomearquivo` para referenciar explicitamente arquivos específicos em prompts com múltiplos arquivos relacionados.
3. Distinguir quais arquivos o Claude Code acessa automaticamente e quais precisam ser referenciados de forma explícita.
4. Adotar a estratégia de solicitar explicação ou plano antes de autorizar qualquer modificação no código-fonte.

Texto Descritivo

Um *prompt* é o texto que o usuário digita no campo de entrada do assistente — em outros termos, é a pergunta ou instrução que orienta o assistente sobre o que se deseja obter. A qualidade da resposta depende diretamente da qualidade do prompt: quanto mais preciso e contextualizado for o texto de entrada, mais útil e específica será a resposta gerada. Pode-se traçar uma analogia com uma consulta médica: se o paciente descreve seus sintomas de forma vaga ("estou me sentindo mal"), o diagnóstico será igualmente vago; se os descreve com precisão ("tenho dor localizada no lado direito do abdômen há 48 horas"), a avaliação pode ser muito mais precisa.

No contexto do Claude Code, a formulação de um bom prompt envolve três elementos fundamentais. Primeiro, **identificar o elemento de código sobre o qual se deseja operar** — por exemplo, mencionar o nome da função, o nome do arquivo e, quando pertinente, as linhas relevantes. Segundo, **descrever o comportamento atual e o comportamento esperado** — o que o código faz hoje e o que deveria fazer. Terceiro, **delimitar o escopo da resposta desejada** — se o participante quer apenas uma explicação, apenas uma sugestão de melhoria ou a aplicação efetiva da mudança. Essa delimitação é especialmente importante: recomenda-se sempre solicitar primeiro uma explicação ou um plano de ação, e apenas em seguida — após revisão — autorizar a modificação do arquivo.

Quanto ao contexto automático, convém registrar o seguinte: o Claude Code acessa automaticamente o conteúdo do arquivo que está aberto no editor e as linhas eventualmente selecionadas pelo usuário. Ele **não** acessa automaticamente arquivos fechados, histórico do Git, variáveis de ambiente ou dependências instaladas. Para projetos que envolvem múltiplos arquivos, o participante pode referenciar explicitamente um arquivo por meio da sintaxe `@nomearquivo` diretamente no corpo do prompt — por exemplo, `"Estou trabalhando`

em `@api.py` e preciso chamar uma função de `@utils.py`. Como fazer o `*import*` correto?".
Recomenda-se essa abordagem para projetos com dois a cinco arquivos principais abertos simultaneamente.

 **Captura de tela sugerida:** Painel do Claude Code com um arquivo Python aberto no editor e um prompt digitado no campo de entrada referenciando o nome do arquivo pela sintaxe `@`.

A seguir, apresenta-se um contraste entre um prompt inadequado e um prompt bem formulado para a mesma situação hipotética, a fim de ilustrar a diferença de qualidade nas respostas obtidas:

Prompt inadequado: "Como melhorar essa função?" — O assistente não sabe a qual função o usuário se refere, nem qual aspecto se deseja melhorar.

Prompt bem formulado: "Vejo que tenho uma função chamada `calculate_total()` no arquivo aberto. Ela itera sobre uma lista de dicionários e soma os valores de uma chave chamada `'price'`. Sugira uma versão mais concisa utilizando a função `sum()` do Python. Não faça a mudança no arquivo ainda – apenas mostre o código sugerido em um bloco de código." — O assistente recebe contexto preciso, instrução clara e restrição explícita sobre o escopo da ação.

Depreende-se, portanto, que a habilidade de formular bons prompts é tão relevante quanto qualquer habilidade técnica de programação. Trata-se de uma competência transferível: os mesmos princípios de precisão, contextualização e delimitação de escopo aplicam-se a qualquer tipo de tarefa que o participante venha a realizar com o assistente, seja revisão de texto acadêmico, análise de planilhas ou elaboração de documentação institucional.

Exercício Prático

Questão de Múltipla Escolha — Avaliação de Prompts (5 minutos)

Analise os quatro prompts abaixo e identifique qual deles está mais bem formulado para obter uma resposta precisa e segura do Claude Code, considerando que o arquivo `relatorio.py` está aberto no editor e contém uma função chamada `gerar_resumo(texto)`:

(A) "Melhore o código."

(B) "Analise a função `gerar_resumo()` no arquivo aberto. Ela recebe um texto longo e retorna apenas as três primeiras frases. Identifique possíveis problemas de desempenho quando o texto de entrada for muito extenso. Não modifique o arquivo – apenas liste os problemas encontrados e sugira alternativas."

(C) "Como funciona Python?"

(D) "Refatore tudo no `@relatorio.py` imediatamente."

Gabarito: Alternativa (B).

Fundamentação: O prompt (B) atende aos três critérios de um bom prompt: identifica o elemento de código (`gerar_resumo()`), descreve o comportamento atual e o cenário problemático (texto muito extenso), e delimita explicitamente o escopo da resposta (apenas listagem de problemas, sem modificação do arquivo). Os demais prompts são excessivamente vagos (A e C) ou excessivamente amplos e sem revisão prévia (D).

Mão na massa 🛠️

Abra (ou crie) qualquer arquivo de texto ou código no VS Code e utilize o prompt base abaixo, adaptando os campos entre colchetes:

Tenho um arquivo aberto chamado [NOME DO ARQUIVO] que contém [DESCREVA BREVEMENTE O CONTEÚDO: uma função, um texto, uma planilha de dados, etc.].
Gostaria de entender [DESCREVA O QUE QUER SABER: como o código funciona, quais são os possíveis problemas, como poderia ser melhorado, etc.].
Por favor, não modifique o arquivo ainda – apenas me explique [ASPECTO ESPECÍFICO].

Módulo 4 — Refatoração Assistida: o ciclo Perguntar → Planejar → Revisar → Aceitar

Sabendo formular prompts contextualizados (Módulo 3), o participante está em condições de dar o próximo passo: realizar modificações reais no código de forma segura, por meio do ciclo de revisão que caracteriza o uso responsável de assistentes de IA em ambientes de desenvolvimento.

Objetivos de Aprendizagem

Ao final deste módulo, o participante deverá ser capaz de:

1. Aplicar o ciclo de quatro etapas — Perguntar, Planejar, Revisar (*diff*) e Aceitar/Rejeitar — em uma tarefa de refatoração de código.
2. Interpretar o visualizador de diferenças (*diff viewer*) do VS Code, identificando as linhas removidas e as linhas adicionadas em uma sugestão de mudança.
3. Solicitar ao assistente uma explicação dos *trade-offs* de uma refatoração antes de autorizá-la.
4. Reconhecer situações em que a sugestão do assistente deve ser rejeitada ou solicitada novamente com parâmetros diferentes.

Texto Descritivo

O termo *refatoração* designa o processo de reescrever ou reorganizar um trecho de código existente sem alterar o seu comportamento externo — ou seja, o código refatorado continua fazendo a mesma coisa, porém de forma mais clara, mais eficiente ou mais fácil de manter. É uma prática rotineira no desenvolvimento de software e igualmente relevante em contextos acadêmicos nos quais scripts são produzidos para análise de dados ou automação de tarefas.

O material-fonte propõe um protocolo de cinco passos para a refatoração segura com auxílio do Claude Code, que pode ser sintetizado em quatro etapas principais para os fins deste curso: **Perguntar** (*Ask*) — solicitar ao assistente uma análise do trecho de código e uma sugestão de melhoria, sem autorizar modificações; **Planejar** (*Plan*) — pedir ao assistente que liste os passos exatos que serão executados e explique os *trade-offs* envolvidos (o que se ganha e o que se perde com a mudança); **Revisar** (*Diff*) — analisar o visualizador de diferenças, que apresenta lado a lado o código original (marcado em vermelho) e o código sugerido (marcado em verde); e **Aceitar ou Rejeitar** (*Accept/Reject*) — aprovar a mudança, rejeitá-la ou solicitar uma nova proposta com parâmetros diferentes.

O *diff viewer* — visualizador de diferenças — é uma ferramenta integrada ao VS Code que exibe, de forma visual e comparativa, as alterações propostas pelo assistente. Linhas precedidas pelo símbolo  e destacadas em vermelho correspondem ao código original que será removido; linhas precedidas por  e destacadas em verde correspondem ao novo código que será inserido. Nenhuma modificação é aplicada ao arquivo enquanto o participante não clicar em *Accept*. Esse mecanismo é análogo à função de "controle de alterações" presente em editores de texto como o Microsoft Word: o documento original permanece intacto até que o revisor aprove explicitamente cada mudança.

 **Captura de tela sugerida:** *Diff viewer* do VS Code exibindo lado a lado o código original (com linhas em vermelho e símbolo ) e o código sugerido (com linhas em verde e símbolo ), com os botões

I *Accept* e *Reject* visíveis na interface.

Convém registrar um ponto de atenção relevante: o assistente de IA pode sugerir mudanças tecnicamente válidas que, no entanto, não sejam adequadas ao contexto específico do projeto — por exemplo, uma refatoração que torna o código mais conciso, porém menos legível para leitores sem experiência avançada em programação. O participante é sempre o árbitro final: a responsabilidade pela aprovação ou rejeição de qualquer alteração recai sobre ele, não sobre o assistente. Recomenda-se, portanto, nunca aceitar uma sugestão sem compreender o que ela faz — e, sempre que necessário, solicitar ao assistente uma explicação adicional antes de prosseguir.

O mesmo ciclo se aplica a tarefas de depuração (*debugging*), que consistem em identificar e corrigir erros em um programa. Ao descrever um erro para o assistente, o participante deve fornecer, sempre que possível: o trecho de código onde o erro ocorreu, a mensagem de erro completa (denominada *stack trace*, que é o registro sequencial das chamadas de função que levaram ao erro) e uma descrição do resultado esperado versus o resultado obtido. Quanto mais precisa for essa descrição, mais eficiente será o diagnóstico do assistente.

Exercício Prático

Ciclo de Refatoração Guiada (15 minutos)

Configuração: Crie um arquivo chamado `exercicio.py` no VS Code com o seguinte conteúdo:

```
def verificar_email(email):
    if email is None:
        return False
    if email == "":
        return False
    if "@" not in email:
        return False
    if "." not in email:
        return False
    return True
```

Passos:

1. Com o arquivo aberto no editor, acesse o painel do Claude Code e digite o seguinte prompt: "Tenho a função `verificar_email()` no arquivo aberto. Ela usa múltiplos `ifs` para validar um e-mail. Sugira uma versão mais concisa. Não modifique o arquivo ainda – apenas mostre o código sugerido."
2. Após receber a sugestão, envie o seguinte prompt: "Antes de aplicar a mudança, explique: (a) o que mudou na lógica; (b) há casos em que a nova versão se comporta de forma diferente da original; (c) quais são os trade-offs entre as duas versões?"
3. Após ler a explicação, envie o prompt: "Entendi a explicação. Por favor, aplique a refatoração na função `verificar_email()`."
4. Analise o *diff viewer* exibido. Leia as linhas removidas (vermelho) e as linhas adicionadas (verde).
5. Decida: o código sugerido está correto e compreensível? Se sim, clique em *Accept*. Se não, clique em *Reject* e solicite uma nova proposta.

Critério de êxito: O participante consegue executar as quatro etapas do ciclo sem aceitar nenhuma mudança que não compreenda. O arquivo final contém a versão refatorada da função, com redução no número de linhas

e manutenção do comportamento original para os casos de teste.

Mão na massa 🛠️

Selecione qualquer função ou trecho de código de um projeto seu (ou de um exemplo público) e aplique o ciclo completo utilizando o prompt base abaixo:

Tenho a função [NOME DA FUNÇÃO] no arquivo aberto.
Ela [DESCREVA O QUE ELA FAZ em uma ou duas frases].
Identifique possíveis melhorias relacionadas a [ASPECTO: legibilidade, desempenho, tratamento de erros, etc.].
Não modifique o arquivo ainda – apenas mostre o código sugerido e explique os trade-offs da mudança.

Módulo 5 — Síntese e Aplicação Integrada

Este módulo não introduz conceitos novos. Seu propósito é consolidar, em uma única atividade integradora, todas as competências desenvolvidas nos módulos anteriores: compreensão do ecossistema (Módulo 1), instalação e configuração (Módulo 2), formulação de prompts contextualizados (Módulo 3) e ciclo de refatoração segura (Módulo 4).

Objetivos de Aprendizagem

Ao final deste módulo, o participante deverá ser capaz de:

1. Aplicar, de forma integrada e sequencial, o ciclo completo de uso do Claude Code — da análise inicial à documentação final de um trecho de código.
2. Avaliar criticamente o resultado de cada etapa do ciclo antes de prosseguir para a seguinte.
3. Identificar, em um cenário concreto, quais dados podem ser compartilhados com o assistente e quais devem ser anonimizados ou omitidos.
4. Documentar, em linguagem acessível, as mudanças realizadas em um trecho de código com auxílio do assistente.

Texto Descritivo

A integração de ferramentas de Inteligência Artificial ao fluxo de trabalho cotidiano é um processo gradual, que se consolida por meio da prática repetida em contextos reais. O presente módulo propõe uma simulação desse processo: o participante receberá um módulo de código com múltiplos problemas e deverá conduzi-lo, com auxílio do Claude Code, desde o estado inicial problemático até um estado final documentado, testável e legível. Esse percurso replica o cenário descrito na Aula 12 do material-fonte, adaptado para o escopo de tempo deste curso.

Observa-se que a atividade integradora não exige conhecimento avançado de programação. O participante pode substituir os exemplos em Python por qualquer outra linguagem ou, se preferir, trabalhar com um texto em linguagem natural que precise ser reorganizado, resumido ou documentado — o ciclo de prompts permanece o mesmo. O que importa, nesta etapa, é a demonstração da competência de conduzir uma interação produtiva com o assistente, tomando decisões conscientes em cada etapa do ciclo.

Recomenda-se que o participante registre, em um arquivo de notas separado, as decisões tomadas em cada etapa: quais sugestões foram aceitas, quais foram rejeitadas e por quê. Esse registro é, em si, uma forma de documentação do processo de aprendizagem e pode ser útil como referência futura.

Exercício Prático — Atividade Integradora (10 minutos)

Contexto: O arquivo abaixo representa um módulo de código com problemas comuns de legibilidade e organização. O participante deverá conduzir, com auxílio do Claude Code, as quatro etapas do ciclo de refatoração.

Arquivo inicial — `notas.py` :

```
def c(l):  
    t = 0  
    for x in l:  
        t = t + x  
    return t / len(l)  
  
def m(l):  
    s = sorted(l)  
    n = len(s)  
    if n % 2 == 0:  
        return (s[n//2 - 1] + s[n//2]) / 2  
    return s[n//2]
```

Sequência de prompts a executar:

Etapas 1 — Perguntar: "Tenho duas funções no arquivo aberto: c(l) e m(l). Analise o código e descreva o que cada função faz. Não modifique nada ainda."

Etapas 2 — Planejar: "Sugira melhorias para tornar o código mais legível: nomes de funções e variáveis mais descritivos, type hints e docstrings em estilo Google. Liste o plano de mudanças antes de aplicar."

Etapas 3 — Revisar e aceitar: "Aplique as melhorias planejadas. Mostre o diff antes de confirmar."

Etapas 4 — Documentar: "Gere um comentário de cabeçalho para o arquivo notas.py descrevendo o que o módulo faz, quais funções contém e um exemplo de uso de cada uma."

Tabela de Critérios de Avaliação:

Critério	Atendido?
O participante executou as quatro etapas do ciclo sem pular a etapa de revisão (diff)	☐ Sim / ☐ Não
Nenhum dado sensível (credenciais, informações pessoais) foi incluído nos prompts	☐ Sim / ☐ Não
O participante leu e compreendeu a explicação do assistente antes de aceitar a refatoração	☐ Sim / ☐ Não
O arquivo final contém nomes de funções legíveis, type hints e docstrings	☐ Sim / ☐ Não
O cabeçalho do módulo foi gerado e inserido no arquivo	☐ Sim / ☐ Não
O participante consegue explicar, com suas próprias palavras, o que cada função faz	☐ Sim / ☐ Não

Critério de êxito: Ao menos cinco dos seis critérios marcados como "Sim", obrigatoriamente incluindo os critérios 1 e 2.

Mão na massa 🛠️

Para consolidar o aprendizado com um tema de sua escolha, utilize o prompt base integrativo abaixo:

Tenho um arquivo chamado [NOME DO ARQUIVO] com [DESCREVA O CONTEÚDO: funções, texto, dados, etc.].
Gostaria de conduzir um ciclo completo de melhoria:

1. Primeiro, analise o conteúdo e identifique os principais problemas ou oportunidades de melhoria.
2. Em seguida, proponha um plano de mudanças para [ASPECTO: legibilidade, clareza, organização, documentação, etc.].
3. Depois, aplique as mudanças e mostre o diff.
4. Por fim, gere um resumo do que foi feito para que eu possa registrar em minhas notas.

Por favor, aguarde minha confirmação a cada etapa antes de prosseguir.

Encerramento

O presente curso percorreu, em cinco módulos sequenciais, o trajeto que vai da compreensão conceitual do ecossistema Claude à aplicação prática do ciclo de refatoração assistida por Inteligência Artificial no ambiente VS Code. Ao concluir esta trilha, o participante adquiriu os fundamentos necessários para interagir produtivamente com o assistente Claude Code em tarefas reais de seu contexto acadêmico ou profissional: desde a formulação de prompts contextualizados até a revisão crítica das sugestões geradas, passando pela configuração segura do ambiente e pelo respeito às boas práticas de privacidade de dados. Recomenda-se a prática regular dessas habilidades em projetos concretos, pois a fluência na interação com assistentes de IA se desenvolve de forma acumulativa — cada sessão de uso aprimora a capacidade de formular perguntas mais precisas e de avaliar respostas com maior segurança.

Para os participantes que desejam aprofundar o aprendizado além dos fundamentos abordados neste curso, indica-se como próximos passos o estudo da geração automatizada de testes unitários (abordada nas Aulas 6 e 7 do material-fonte), a exploração do Model Context Protocol (MCP), que permite ao assistente acessar fontes externas como repositórios Git e bancos de dados (Aula 11), e a prática de automação de tarefas repetitivas por meio de scripts auxiliares (Aula 9). A documentação oficial da Anthropic, disponível em <https://docs.anthropic.com>, constitui referência atualizada para todos esses tópicos e deve ser consultada para verificação de eventuais atualizações nas funcionalidades da extensão.

Síntese: O uso produtivo de um assistente de Inteligência Artificial não reside na delegação irrestrita de decisões à ferramenta, mas na construção de um diálogo preciso, informado e criticamente supervisionado — em que o profissional humano permanece responsável por cada escolha aprovada.